Modification d'un entrepôt de données

Gattano César

9 octobre 2026

Avant-propos

Structure du rapport

Ce rapport fournit les explications sur ce qui a été mis en place dans le cadre du livrable E6 relatif à la maintenance d'un entrepôt de données de la certification "expert en infrastructures de données massives" chez Simplon. Un travail technique étant toujours améliorable, l'auteur a reporté en annexe différentes pistes d'amélioration rencontrées durant la veille si le travail devait être renforcé.

Dans cette même annexe, l'auteur reporte les liens vers les sources dont il s'est inspiré pour aborder les différentes points techniques de ce livrable. Ces deux parties de l'annexe suivent la même structure que la structure du rapport

Usage de l'IA générative

Ce rapport a été rédigé en usant de l'intelligence artificielle. Par soucis de transparence, les parties purement rédigées par l'IA sont signalées via une couleur de police bleu sombre. L'auteur certifie que l'ensemble du contenu généré par l'IA a été revu et validé par ses soins.

Voici un exemple : Je suis un assistant IA conçu pour vous aider dans la rédaction de votre rapport. Je génère du texte structuré et cohérent, que vous pouvez ensuite affiner. Mon rôle est de vous fournir un cadre de travail, tout en vous laissant le contrôle final. Ensemble, nous pouvons créer un document clair et professionnel.

En outre, l'auteur indique que l'IA générative a également été utilisée en auto-complétion durant la rédaction de ce rapport. Cette fonctionnalité ayant été utilisée tout au long du rapport, aucun indicateur n'a été mis en place pour souligner cet aspect, si ce n'est cette déclaration.

Table des matières

1 Contexte **4**

2 Préparation de la maintenance **4**
2.1 État initial de l'entrepôt de données 4
2.2 Planification de la maintenance 5
2.3 Suivi du service 5
2.4 Backup régulier 6
2.4.1 Procédure de sauvegarde 6
2.4.2 Procédure de restauration 6

3 Opération de maintenance **8**
3.1 Techno `DBT` 8
3.2 Ajout d'un datamart 8
3.2.1 Intégration d'une nouvelle source de données 8
3.2.2 Modèle de staging des données brutes 8

3.2.3 Création de la table des faits associés aux nouvelles données . . . 9
3.3 Tests et documentations DBT . . . 10
3.3.1 Tests génériques . . . 10
3.3.2 Tests singuliers . . . 10
3.3.3 Écriture de la documentation . . . 10
3.3.4 Génération de la documentation . . . 10

4 Gestion des accès et procédure d'ajout d'un nouveau rôle 12
4.1 Audit de la situation avant toute opération . . . 12
4.2 Ajout d'un role . . . 12
4.3 Attribution des privilèges . . . 12
4.4 Tests de validation . . . 13
4.5 RGPD . . . 13

5 Variation d'une dimension 15
5.1 SCD de Ralph Kimball . . . 15
5.2 Les SCD dans DBT . . . 16

6 Scalabilité 17
6.1 Procédure d'ajout d'un datamart . . . 17
6.2 Augmentation de l'espace de stockage . . . 18

7 Conclusion 20

A Exemple d'une fiche de traitament (RGPD) 21

B Difficultés rencontrées 21
B.1 Préparation de la maintenance . . . 21
B.1.1 À propos du backup régulier . . . 21
B.2 Opération de maintenance . . . 22
B.2.1 À propos de l'ajout d'un datamart . . . 22
B.2.2 À propos des tests et documentations DBT . . . 22
B.3 Gestion des accès et procédure d'ajout d'un nouveau rôle . . . 22
B.4 Variation d'une dimension . . . 22
B.4.1 À propos des SCD dans DBT . . . 22

C Liens utiles 23
C.1 Préparation de la maintenance . . . 23
C.1.1 À propos du suivi de service . . . 23
C.1.2 À propos du backup régulier . . . 23
C.2 Opération de maintenance . . . 23
C.2.1 À propos de l'ajout d'un datamart . . . 23
C.2.2 À propos des tests et documentation DBT . . . 23
C.3 Gestion des accès et procédure d'ajout d'un nouveau rôle . . . 23
C.3.1 À propos de l'attribution des privilèges . . . 23
C.3.2 À propos des tests de validation . . . 23
C.3.3 À propos du RGPD . . . 24
C.4 Variation d'une dimension . . . 24
C.5 À propos des SCD dans DBT . . . 24
C.6 Scalabilité . . . 24
C.6.1 À propos de l'ajout d'une datamart . . . 24

D Pour aller plus loin **25**
D.1 Préparation de la maintenance . . . 25
D.1.1 À propos du suivi de service . . . 25
D.1.2 À propos du backup régulier . . . 25
D.2 Opération de maintenance . . . 25
D.2.1 À propos de l'ajout d'un datamart . . . 25
D.3 Gestion des accès et procédure d'ajout d'un nouveau rôle . . . 25
D.3.1 À propos de l'attribution des privilèges . . . 25
D.3.2 À propos du RGPD . . . 25
D.4 Variation d'une dimension . . . 26

1 Contexte

Je suis data engineer junior au sein de *ShopFlow*, une ESN spécialisée dans les solutions data pour le secteur e-commerce. Mon client, la société *VenteRapide*, exploite une boutique en ligne généraliste présente sur le marché français depuis 2021.

VenteRapide dispose d'un entrepôt de données (data warehouse) opérationnel, construit sur **PostgreSQL** et alimenté via **DBT**. Cet entrepôt centralise les données de commandes, de produits, de clients et de visites du site, et est utilisé quotidiennement par l'équipe analytique pour ses reportings.

Dans le cadre d'une évolution de l'activité, VenteRapide me confie la mission de maintenir et faire évoluer cet entrepôt en conditions opérationnelles.

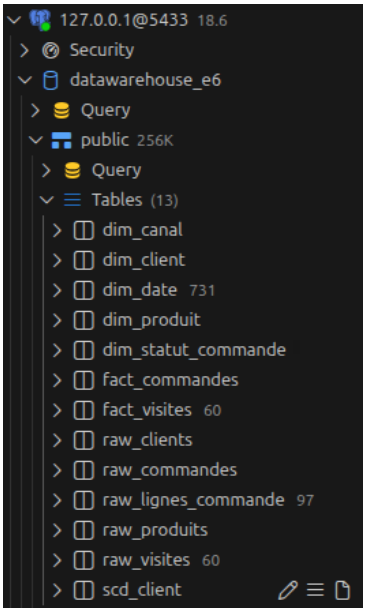


FIGURE 2 – Entrepôt initial

2 Préparation de la maintenance

2.1 État initial de l'entrepôt de données

L'entrepôt initial contient :

- 5 tables de données brutes : `clients`, `commandes`, `lignes_commandes`, `produits`, `visites`,
- 5 tables de dimensions : `canal`, `client`, `date`, `produit`, `statut_commandes`,
- 2 tables de faits : `commandes`, `visites`.
- 1 table SCD type 2 : `client`.

Le lignage des données est assuré par **DBT**, qui orchestre les transformations et les chargements. La figure 1 montre le lignage des données dans l'entrepôt initial.

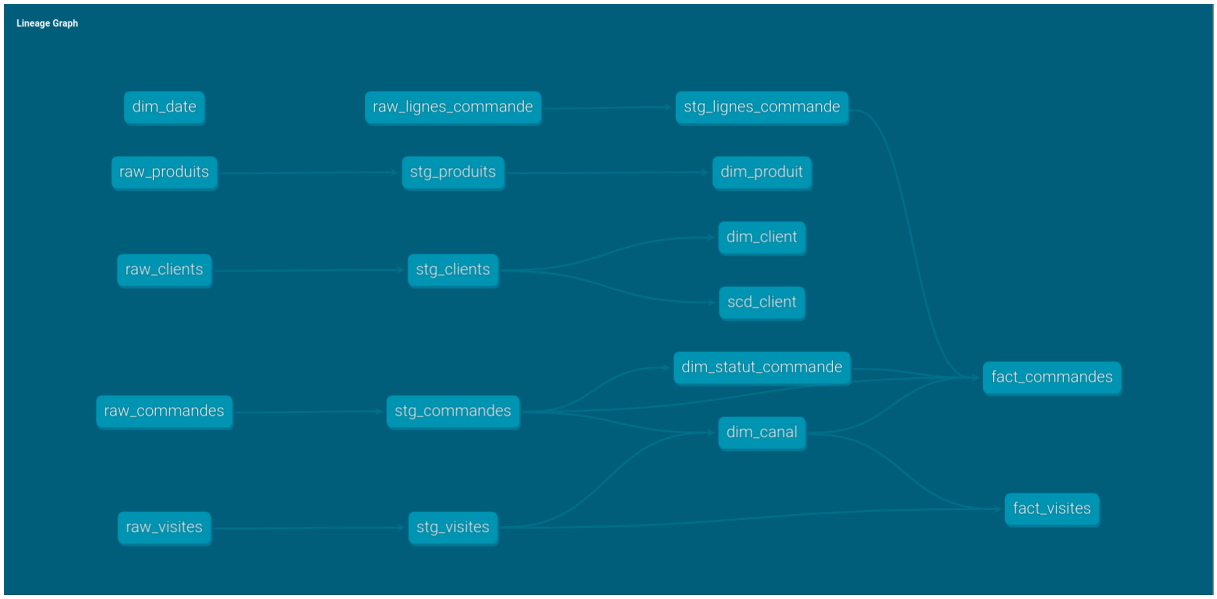
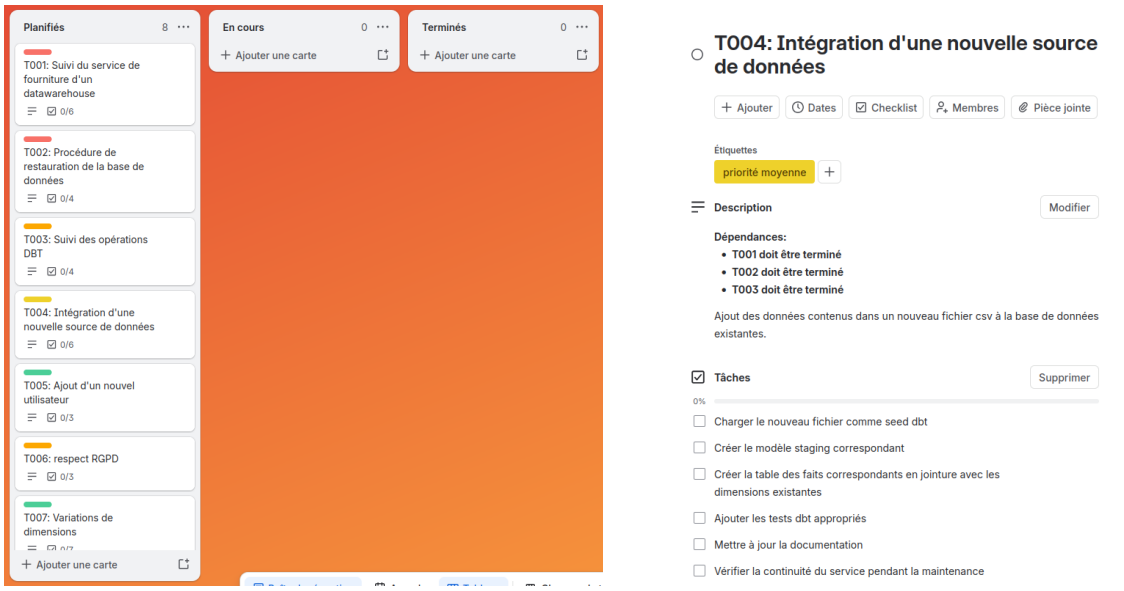


FIGURE 1 – Lignage des données dans l'entrepôt initial

2.2 Planification de la maintenance

La maintenance de l'entrepôt de données est planifiées grâce à un système de tickets. La vue d'ensemble et un exemple de tickets sont présentés en figure 3.



(a) Vue du tableau (b) Exemple de ticket de maintenance

FIGURE 3 – Système de tickets de maintenance.

2.3 Suivi du service

Un SLA (*Service Level Agreement*) est un engagement formel entre un fournisseur de service et ses utilisateurs, définissant le niveau de qualité de service attendu, mesurable par des indicateurs précis, les SLI (Service Level Indicators). Il répond à trois questions : quoi (quel service), combien (quel niveau de performance), et que se passe-t-il en cas de non-respect.

Dans le cas d'une base de données, un SLA pourrait inclure des engagements sur :

- **La disponibilité du service sur une période donnée** : par exemple, un engagement de 99,9% de disponibilité mensuel.
- **Le temps de traitement des incidents** : délai de prise en charge et délai de résolution pouvant être fonction du niveau de criticité
- **La planification de fenêtre de maintenance** : plages de temps où la contrainte de disponibilité peut être levée.
- **RPO (*Recovery point objective*) et RTO (*Recovery time objective*)** : les indicateurs précisant la quantité maximale de perte de données et le temps maximal de restauration acceptable, en cas de rupture de service.
- **Performance** : temps de réponse des requêtes, nombre de transactions par seconde, etc.

Dans le cadre de ce projet, je fais le choix de mettre en place les SLI suivants :

1. Mesure (booléenne) de la santé du conteneur `PostgreSQL`, i.e. simple tentative de connexion régulière pour vérifier que la base de données est accessible.
2. Mesure de la latence d'une requête simple sur les tables de faits de la base de données (tables de haute valeur pour les équipes métiers).

Le script `set_monitoring_db.sh` dans le dossier `scripts/monitoring` permet de créer la base des données de monitoring et l'utilisateur associé, ainsi que les droits nécessaires via des requêtes

SQL envoyées au sein du conteneur **PostgreSQL**. Le script `monitor.py` à la racine contient le code de surveillance de la base de données qui écrit à intervalle régulier les mesures décrites ci-dessus dans une table. Enfin le script `streamlit/ monitor_viz.py` permet de visualiser ces mesures de latence à l'aide d'un tableau de bord streamlit (figure 4).

liens utiles / pour aller plus loin

2.4 Backup régulier

La stratégie de back-up mis en place repose sur les outils **pg_dump** et **pg_dump_all** pour la sauvegarde, et **psql** et **pg_restore** pour la restauration.

Il faut distinguer deux types de backup :

1. le backup complet qui reconstruira l'environnement de production (roles, privilèges, bases de données, etc.) dans sa globalité, et
2. le back-up partiel qui va sélectionner les entités particulières de la base **PostgreSQL** à sauvegarder.

pg_dump tombe dans la catégorie du back-up partiel puisqu'il va permettre de sauvegarder les données d'une base de données (mais ne considèrera pas par exemple la création des rôles ayant des droits sur cette table car ce sont des entités globales). **pg_dumpall** tombe dans la catégorie du back-up complet (bien que certaines options permettent de limiter son périmètre).

2.4.1 Procédure de sauvegarde

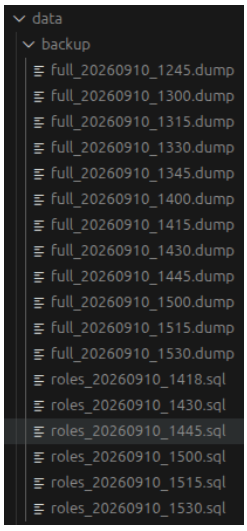


FIGURE 5 – Backup 15min, rétention 3h

En guise de stratégie de sauvegarde et pour expérimenter les deux approches, nous allons procéder comme suit :

- Procédure de sauvegarde complète que l'on restreint à l'extraction des rôles et privilèges, avec **pg_dumpall**
- Procédure de sauvegarde partielle de la seule base de données `datawarehouse_e6` , avec **pg_dump**

Un premier script `scripts/backup/ set_backup_roles.sh` permet la première sauvegarde et un deuxième `scripts/backup/ set_backup_db.sh` permet la seconde. À savoir qu'il existe deux formats possibles pour les fichiers-dump : ficher-sql contenant les commandes **SQL** pour reconstruire la sauvegarde à l'identique (via **psql**) ou un format dédié compréssé (grâce à **zlib**) utilisable par **pg_restore** pour la reconstruction.

Ces scripts permettent également d'opérer la période de rétention des fichiers en appliquant le temps de conservation des fichiers-sql/dump. C'est ensuite un **cron** qui permet la régularité des opérations de back-up via la planification de l'exécution de ces deux scripts.

Pour l'exercice d'apprentissage, une éxecution toutes les 15 minutes avec un période de rétention de 3 heures a été l'option choisie (voir fig. 5. En situation opérationnelle, une exécution quotidienne serait plus adaptée avec une retention d'une semaine ou d'un mois.

2.4.2 Procédure de restauration

La stratégie de restauration que nous allons voir va permettre de reproduire la base de données `datawarehouse_e6` tout en conservant les accès et privilèges des différents utilisateurs. Afin de tester la procédure, un conteneur jetable sera mis en place afin que les deux tables (originale et restaurée) puissent co-exister.



FIGURE 4 – Screenshot du tableau de bord de monitoring de la base de données

Afin d'assurer le succès de l'opération, il est nécessaire de reproduire les rôles dans le conteneur dédié à la restauration, puis de restaurer la base de données `datawarehouse_e6` (l'ordre est important). Le script `scripts/backup/ test_backup_db.sh` opère cela après la mise en place d'un conteneur.

Vient ensuite une phase de validation avec le lancement de plusieurs commandes **SQL** pour afficher certaines métriques :

- Liste des rôles restaurés
- Liste des droits sur la table `fact_commandes`
- Nombre d'entrées dans chaque table

Ces sorties sont à comparer avec les mêmes commandes exécutées sur le conteneur original pour s'assurer qu'il n'y a aucune différence.

Enfin, les tests **DBT** sont une validation supplémentaire, plus approfondie permettant de s'assurer qu'il n'y a pas eu de corruption des données, ou de perte de données entraînant une violation des règles mises en place dans les modèles des tables et de leurs relations.

Difficultés rencontrées / liens utiles / pour aller plus loin

3 Opération de maintenance

3.1 Techno DBT

DBT s'intègre dans l'approche ELT (*Extract, Load, Transform*) qui se démocratise avec l'avénement du cloud et en particulier des technologies de datawarehouse et la réduction du coût de stockage qui permet de ne pas dépendre de machines *on-premise* et de leur gestion. Cette approche déploie la puissance d'un datawarehouse pour la transformation en les chargeant telles qu'elles sont au préalable. Les données sont ainsi plus facilement et rapidement gérables et traçables. **DBT** fournit une solution de transformation de données propres et prêtes pour l'analyse, en poussant l'automatisation et la collaboration sur la modélisation et le traitement des données. Le principe derrière **DBT** et de reproduire les bonnes pratiques en développement d'applications pour le développement de modèles analytiques de données.

3.2 Ajout d'un datamart

3.2.1 Intégration d'une nouvelle source de données

Le fichier **`raw_retours.csv`** a été ajouté aux `seeds` **DBT**, pour l'intégrer comme une nouvelle source de données. En parallèle, un fichier **`schema.yml`** a été générée pour fournir la description de tous les fichiers-csv présent, à des fins de documentation.

Ensuite la commande :

```
dbt seed
```

va s'emparer du nouveau fichier-csv et créer une table dans la base de données.

3.2.2 Modèle de staging des données brutes

Le modèle de staging selon **DBT**, est un modèle où peu de transformations sont opérées à partir des données des sources. Il est recommandé de les matérialiser en vues plutôt qu'en table et servent essentiellement à renommer et caster les colonnes, et à effectuer des tests sur ces données pour les normaliser (unicité, non-null, valeurs autorisées, ...).

Dans `models`, un fichier **`stg_retours.sql`** a été créé pour modéliser la table de staging des données de retours.

```sql
with source as (
    select * from {{ ref('raw_retours') }}
),

renamed as (
    select
        retour_id::integer,
        command_id::integer,
        produit_id::integer,
        client_id::integer,
        date_retour::date,
        motif,
        montant_rembourse::numeric(10,2)
    from source
)
```

En complément, le fichier **schema.yml** a été complété pour inclure la documentation et les tests d'unicité, non-nullité et de valeurs autorisées sur les différents champs de la table `retours`.

Pour finir la commande :

```
dbt build --select stg_retours
```

Permet la génération de la vue et l'exécution des tests programmés dans le fichier **schema.yml**.

3.2.3 Création de la table des faits associés aux nouvelles données

La table des faits à intégrer respecte un schéma en étoile déjà en place au sein de **DBT**. Ainsi, la table des faits `fact_retours` doit être reliée aux tables de dimensions existantes (on parle de dimensions conformes).

Dans `models/facts`, un fichier **fact_retours.sql** a été créé pour modéliser la table. Cette fois, la table est matérialisée comme une table car elle est destinée à être interrogée par les équipes métiers et analystes. Entre autres, ce fichier contient les jointures de tables entre deux modèles de staging et les tables de dimensions existantes :

```sql
select
    r.retour_id,
    r.commande_id,
    to_char(c.date_commande, 'YYYYMMDD')::integer   as date_commande_id,
    to_char(c.date_expedition, 'YYYYMMDD')::integer as date_expedition_id,
    to_char(c.date_livraison, 'YYYYMMDD')::integer   as date_livraison_id,
    to_char(r.date_retour, 'YYYYMMDD')::integer   as date_retour_id,
    dc.client_id,
    dp.produit_id,
    r.motif,
    l.montant_ligne as montant_paye,
    r.montant_rembourse,
    case
        when c.date_livraison is not null
        then (r.date_retour - c.date_livraison)
        when c.date_commande is not null
        then (r.date_retour - c.date_commande)
        else null
    end as delai_retour_jours
from retours r
left join commande c       on r.commande_id = c.commande_id
left join lignes l         on r.commande_id = l.commande_id and r.produit_id = ...
    l.produit_id
left join dim_client dc    on r.client_id = dc.client_id
left join dim_produit dp   on r.produit_id = dp.produit_id
```

En complément le fichier **schema.yml** a été complété pour inclure la documentation, les tests d'unicité, non-nullité et de valeurs autorisées ainsi que les relations avec les tables de dimension.

Pour finir la commande :

```
dbt build --select fact_retours
```

permet la matérialisation de la table et l'exécution des tests programmés dans le fichier `schema.yml`.

Difficultés rencontrées / liens utiles / pour aller plus loin

3.3 Tests et documentations DBT

3.3.1 Tests génériques

Ils sont intégrés aux fichiers `schema.yml` et permettent de vérifier l'unicité, la non-nullité, l'appartenance à des valeurs autorisées, et la validité des relations entre modèles de données.

3.3.2 Tests singuliers

En complément des tests génériques, il est possible d'implémenter des tests singuliers propres à la logique métier à l'aide de fichier SQL. Par défaut un test DBT valide est une requête SQL qui retourne 0 ligne de données.

Un exemple de test singulier a été mise en place dans le dossier `tests` :
`asser_fat_retours__delai_retours_jours_positive.sql`

```
select
    retour_id,
    delai_retour_jours
from {{ ref('fact_retours') }}
where delai_retour_jours IS NOT NULL AND delai_retour_jours < 0
```

Ce test permet de s'assurer que le délai de retours n'est jamais négatif (ce serait une aberration).

3.3.3 Écriture de la documentation

Comme mentionné précédemment, la première documentation DBT est intégrée aux fichiers `schema.yml`. Elles ont étaient complétées à chaque étape de la maintenance. Leur intégration peut se faire directement dans les fichiers ou alors via une référence à un bloc de documentation dans un fichier-md existant. Un exemple de documentation séparée est le fichier `models.md` dans `models/` :

```
{% docs dim_client %}
Dimension client issue du systeme e-commerce.
Contient les informations de chaque client avec son segment (Premium/Standard),
sa localisation et sa date d'inscription.
Utilisee en jointure avec fact_commandes et fact_visites via client_id.
{% enddocs %}

{% docs dim_produit %}
Dimension produit du catalogue e-commerce.
Contient les caracteristiques de chaque produit : categorie, marque, prix,
cout d'achat et marges calculees.
{% enddocs %}

...
```

3.3.4 Génération de la documentation

En DBT, la documentation générée, basée sur la documentation écrite (voir section précédente), l'est à l'aide de la commande :

```
dbt docs generate
```

Cette commande va générer un site web statique dans le dossier `target` qui peut être visualisé avec la commande :

```
dbt docs serve
```

Ce site permet de parcourir les documentations spécifiques de chaque seed et modèle (colonnes, types, descriptions, contraintes, tests, voir figure 6). Surtout, il permet de visualiser le lignage des données et les relations entre les modèles de données. La figure 7 montre le lignage des données après l'intégration de la nouvelle source de données et la création de la table des faits associée.

Difficultés rencontrées / liens utiles

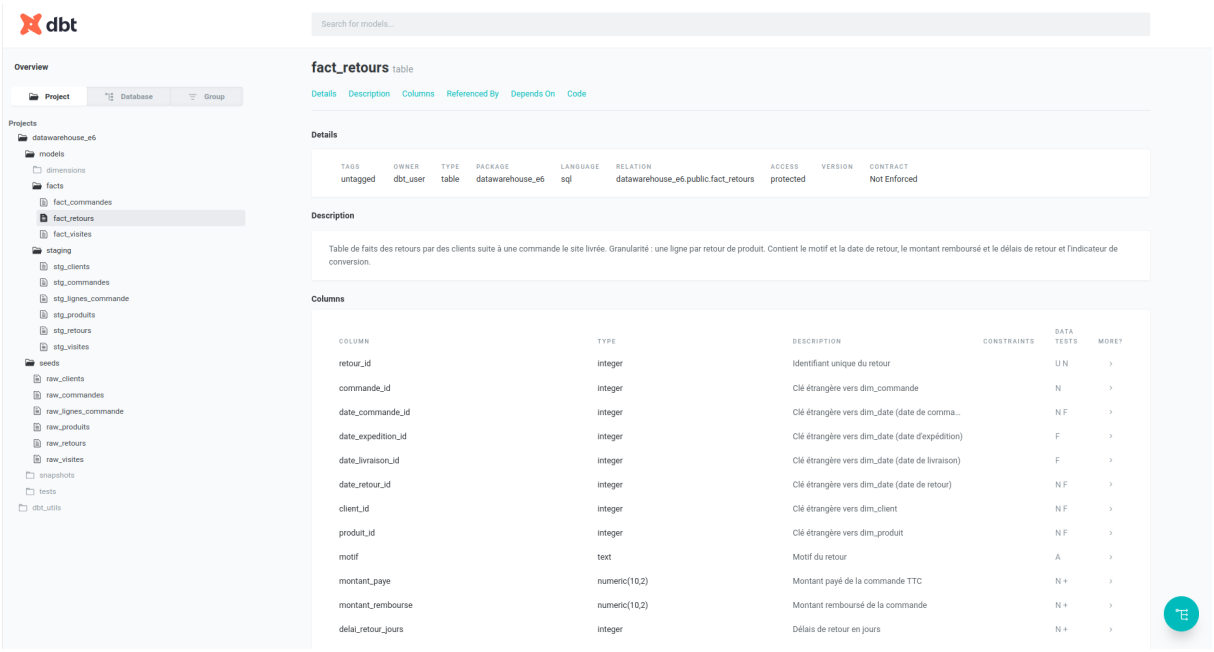


FIGURE 6 – Page de documentation générée par **DBT** pour la table `fact_retours`

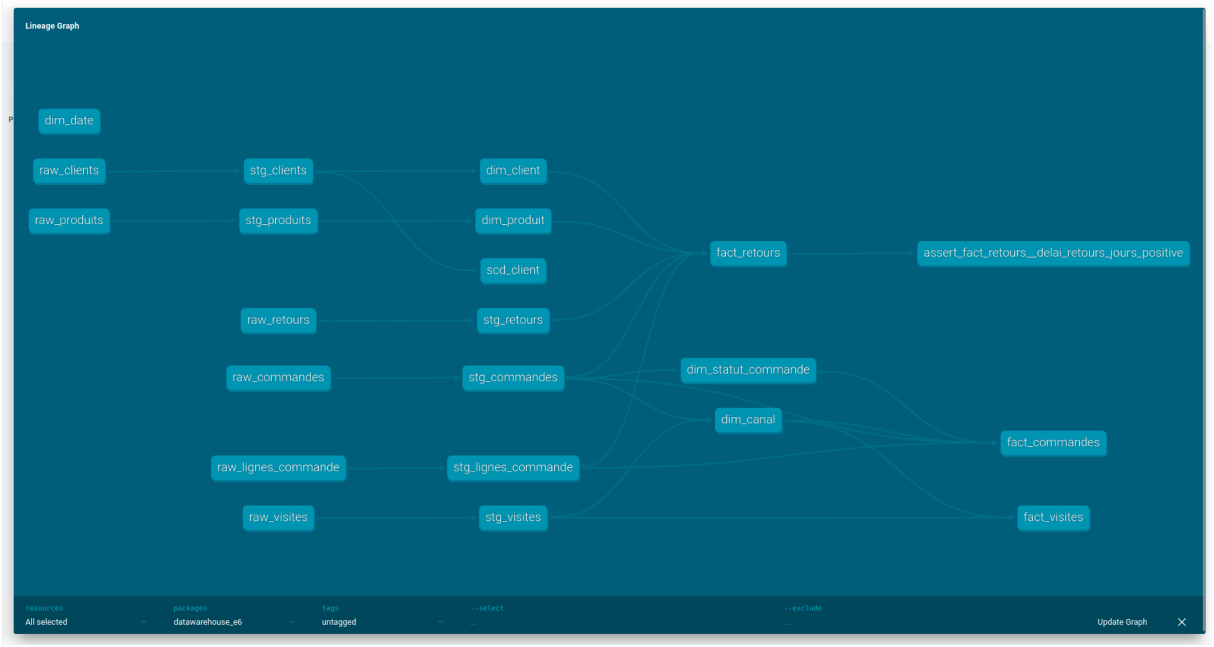


FIGURE 7 – Lignage des données dans l'entrepôt après maintenance

4 Gestion des accès et procédure d'ajout d'un nouveau rôle

4.1 Audit de la situation avant toute opération

Le script `scripts/access/ 00_audit.sql` est un ensemble de requêtes **SQL** qui permettent de récupérer toutes les informations pertinentes sur les rôles et privilèges existants dans la base de données. Il est exécuté via le script `scripts/access/ 00_audit.sh`.

```
SELECT version();
```

récupère la version de **PostgreSQL** de la base de données en cours d'utilisation.

```
\du
```

récupère la liste des rôles existants sur la base de données, ainsi que leurs attributs.

```
\dn+
```

récupère la liste détaillée des schémas.

```
\dp public.*
```

récupère la liste des privilèges et règles sur les objets du schéma public.

```
\ddp
```

récupère la liste des privilèges par défaut sur les objets.

```
SELECT datname, datacl FROM pg_database WHERE datname = current_database();
```

récupère la liste des privilèges sur la base de données en cours d'utilisation. Pour information, le motif `[<role1>]=[C][T][c]/<role2>` signifie que le rôle 1 a les privilèges de création de nouveaux schémas au sein de la base de données (C), de création de tables temporaires lors de l'utilisation de la base de données (T) et de connexion à la base de données (c), et que ces privilèges ont été accordés par le rôle 2.

4.2 Ajout d'un role

Un script `scripts/access/ 01_create_role.sql` a été créé pour ajouter un nouveau rôle à la base de données.

```
CREATE ROLE reporting NOLOGIN
```

Ce rôle **reporting** est un groupe destiné à regrouper les utilisateurs ayant besoin d'accéder aux données pour le reporting.

```
CREATE ROLE <role_name> LOGIN PASSWORD <password> IN ROLE reporting
   NOSUPERUSER NOCREATEDB NOCREATEROLE CONNECTION LIMIT 5'
```

Ce rôle **<role_name>** est un utilisateur qui peut se connecter à la base de données avec le mot de passe `<password>`. Les privilèges de création lui sont retirés, et on limite le nombre de connexions simultanées.

4.3 Attribution des privilèges

Un script `scripts/access/ 02_grant_privileges.sql` a été créé pour attribuer les privilèges nécessaires au rôle **reporting** (et donc à ses utilisateurs). Il s'éxecute via le script `scripts/access/ 02_grant_privileges.sh`. La commande **SQL** `GRANT` est utilisée de différentes manières pour accorder des droits de connexions, d'usage et de sélection sur différentes tables. Un cas particulier est la table `dim_client` contenant des données personnelles. L'application du RGPD impose de restreindre l'accès à certaines données pour le rôle **reporting**.

Aussi intéressantes soient-elles à découvrir, la gestion des attributions via **psql** ne sont malheureusement pas résilients à un projet **DBT**. En effet, lorsqu'une commande `dbt build` est exécutée, les objets sont détruits durant le processus avant d'être récréés. Les privilèges étant apposés sur les objets, ils disparaissent avec eux. Pour assurer la persistence des privilèges, il est alors nécessaire de les ajouter dans les fichiers de configuration du projet **DBT**.
Exemple au niveau de **dbt_project.yml** :

```
models:
  datawarehouse_e6:
    facts:
      +materialized: table
      +grants:
        select: ['reporting']
```

Exemple au niveau des modèles, **schema.yml** :

```
models:
  - name: dim_produit
    description: "{{ doc('dim_produit') }}"
    config:
      grants:
        select: ['reporting']
```

Pour une solution plus granulaire, comme sur `dim_client` qui contient des données personnelles, il est nécessaire d'utiliser un post-hook directement dans l'en-tête du fichier du modèle **dim_client.sql** :

```
{{ config(
    materialized='table',
    post_hook="grant select (client_id, ville, code_postal, pays, segment,
                             date_inscription, actif)
               on {{ this }} to reporting"
) }}
```

Difficultés rencontrées / liens utiles / pour aller plus loin

4.4 Tests de validation

Les tests de validation de la création du nouvel utilisateur sont faits grâce au scrypt **python** `scripts/access/tests/` **test_reporting_access.py**. Ce script utilise **Pytest** pour exécuter une série de tests unitaires couvrant une large gamme de scénarios d'accès à la base de données et encode le comportement attendu dans le cas de l'utilisateur **reporter** du groupe **reporting**.
La commande pour lancer ces tests est :

```
pytest scripts/access/tests -v -rx
```

Difficultés rencontrées / liens utiles

4.5 RGPD

Dans le cadre de la RGPD, les données sensibles identifiées se situent toutes dans la dimension `client` :

- `nom`, `prenom` et `nom_complet` sont indéniablement des données identifiantes
- `email` est également une donnée identifiante sensible

- `date_naissance`, `age` ainsi que `ville`, `code_postal` ne sont pas identifiant pris isolément mais leur couplage peut amener à une situation d'identification de personnes. À ce titre, ces données doivent également être protégées.

La stratégie de gestion des accès à ces données a été mis en place précédemment en n'autorisant pas la lecture de ces colonnes pour le rôle **`reporting`**. Certaines données pourraient être transformées pour être diluées dans une vue des données anonymisées issue de `dim_client`.

Registre des activités de traitements

Ce registre est décrit dans l'article 30 du RGPD. Il est de la responsabilité du responsable de traitement de recenser l'ensemble des traitements mis en oeuvre au moyen d'une fiche du registre par traitement. Un traitement est une activité qui a une finalité. Le nom et les coordonnées de l'organisme, ainsi que le nom et les coordonnées du DPO (*Délégué à la Protection des Données*) si un tel est désigné. Une fiche doit comporter :

- Le nom et les coordonnées du responsable conjoint du traitement.
- La ou les finalités du traitement.
- Les catégories des personnes concernées.
- Les catégories de données personnelles (ex : identité, situation familiale, économique, données bancaires, de connexion, de localisation, etc.).
- Les catégories de destinataires auxquels les données à caractère personnel ont été ou seront communiquées.
- La durée de conservation des différentes catégories de données.
- Une description générale des mesures de sécurité techniques et organisationnelles mises en oeuvre.

Procédure de tri/purge

Les données personnelles sont présentes à plusieurs endroits :

- `seed` : le fichier **`raw_clients.csv`** est chargé initialement.
- `raw` : la table `raw_clients` contient ces mêmes données sans modification.
- `staging` : la vue `stg_clients` contient ces données normalisées.
- `dimension` : la table `dim_clients` recensent ces données dans le schéma final en étoile prêtes à l'emploi pour analyse.
- `back-up` : les fichiers **`full_yyyymmdd_hhmm.dump`** contiennent des copies de ces données à l'instant de l'opération de back-up.

La procédure de tri/purge devra alors effacer une donnée sur l'ensemble de ces supports. **DBT** est à nouveau très utile pour effacer une donnée à la source et la propager à travers le DAG. Après anonymisation de la donnée (e.g `nom, prenom, nom_complet = anonymous`, `email=anonymous@email.fr`, `date_de_naissance=<année_conservée>-01-01`, `code_postal=<dep_conservée>999`), la commande suivante va propager cette anonymisation à travers les couches :

```
dbt build
```

Attention certaines règles de tests peuvent être violées à la suite d'une purge (test de non-nullité ou d'unicité par exemple). Deux cas de figure pour déclencher la procédure de purge existent :

- ✦ Une règle automatique sur la suppression des données personnelles doit être programmée à l'issue d'une période d'inactivité. Dans ce cas, la mise en place d'un **`cron`** sera adaptée pour vérifier régulièrement si chaque utilisateur est toujours dans le cas prévu de conservation de ses données personnelles.
- ✦ Des demandes explicites de suppression des données personnelles peuvent survenir. Dans ce cas, le lancement de la purge se fait de façon manuelle.

En ce qui concerne la présence des données personnelles dans les back-up, ceux-ci sont déjà supprimées automatiquement dans des délais court termistes. La procédure de purge stipule alors simplement qu'une purge doit être réexécutée en cas d'une opération de restauration de la base de données.

liens utiles / pour aller plus loin

5 Variation d'une dimension

5.1 SCD de Ralph Kimball

Ralph Kimball a introduit le schéma en étoile dans la modélisation des données. Dans cette modélisation, les tableaux de dimensions sont peu variables par rapport aux tableaux de faits. Mais ils ne sont pas pour autant invariables, on dit qu'ils changent lentement, appelés SCD (*Slow Changing Dimension*).

Les types de SCD réflètent différentes manières de traiter un tableau de dimensions. Ils permettent de mettre à jour ces tableaux suivant un protocole propre et gèrent l'historique de façon différente.

SCD type 1

Si un enregistrement d'un tableau de dimensions est modifié, les valeurs existantes de cet enregistrement sont écrasées par les nouvlles (pas d'ajout d'une nouvelle ligne). S'il l'enregistrement n'est pas présent, il est inséré (ajoutant une nouvelle ligne). On parle d'"upserting". Ce protocole est utilisé lorsqu'il n'y a pas de besoin de conserver un historique des données. Il n'y a alors pas d'enregistrement en double. La table de dimension renseigne sur l'état le plus récent des données, rendant les requêtes simples mais limités.

id_client	prénom	nom	email
1	Bob	Dylan	bob.dylan@un-email.com

SCD type 2

Dans un SCD type 2, lorsqu'un enregistrement de la table de dimensions évolue, les valeurs composant l'enregistrement précédent sont conservées. En pratique, une (ou plusieurs) colonne(s) supplémentaire(s) est(sont) ajoutée(s) comme attribut(s) de l'état de l'enregistrement ou sa période de validité. La solution la plus répandue est d'ajouter deux colonnes dates fournissant l'intervalle de validité de l'enregistrement. Ce type SCD est le plus adapté lorsqu'on a besoin d'effectuer une analyse temporelle sur les données, offrant une profondeur pour des prises de décision côté métier.

id_produit	nom	prix_unitaire	valid_from	valid_to
1	Super-PC	999.99	2022-01-01	2024-01-01
1	Super-PC	1099.99	2024-01-01	2026-01-01
1	Super-PC	1145.99	2026-01-01	*NULL*

SCD type 3

Avec le SCD type 3, on dispose d'un entre-deux. La solution est de garder, pour une ou plusieurs colonnes de la table des dimensions, deux valeurs (courante et précédente). C'est une solution adaptée pour une dimension très peu variable avec un besoin de garder l'historique tout en conservant l'unicité des enregistrements. On duplique à la place la liste de colonnes dont on souhaite l'historique au sein même

d'un enregistrement. En revanche, au delà de deux modifictions d'un enregistrement, il commence à y avoir perte de données, d'où l'intérêt de ne l'appliquer qu'à des dimensions rarement variables.

id_user	password_current	password_previous	updated_at
1	un_j0l1_mOt2P4$$	pwd_A_ne_pas_R3ut1l1sé	2026-01-01

5.2 Les SCD dans DBT

Focus `scd_client` (type 2)

Une table de dimension SCD de type 2 est mise à disposition dans ce projet : `scd_client`. Dans **DBT**, un SCD type 2 est appelé un *snapshot*. Celui-ci a été mise en place à l'aide d'une méthode dépréciée puisqu'il est défini directement dans le `snapshots/ scd_client.sql`, à l'aide de commandes **jinja**. Aujourd'hui il est recommandé de déclarer un snapshot dans un fichier `snapshots/ snapshots.yml` comme suit :

```
snapshots:
  - name: scd_client_new
    relation: ref('stg_clients')
    config:
      schema: snapshots
      unique_key: client_id
      strategy: check
      check_cols: [ville, code_postal, segment]
```

La stratégie choisie est `check`. Elle compare une liste de colonnes entre ses valeurs courantes et ses valeurs historiques pour déterminer si un enregistrement a changé. À noter qu'il existe une autre stratégie, `timestamp` qui utilise une colonne `updated_at` pour déterminer si un enregistrement a changé. Cette dernière est recommandée lorsqu'elle est possible. Dans le cadre de l'exercice, les colonnes choisies pour la stratégie de snapshot sont `ville`, `code_postal` et `segment`. Des changements sur les autres colonnes ne déclencheront pas d'historisation.

Pour connaître les snapshots du projet **DBT**, la commande suivant permet de les lister :

```
dbt ls --resource-type snapshot
```

Exécution d'une évolution de `scd_client`

Supposons qu'un client renseigne un déménagement. Pour cela dans le `seed`, nous allons modifier les valeurs de `ville` et `code_postal` du client. On modifie directement l'enregistrement associé dans le fichier `raw_client.csv` et on exécute les commandes :

```
dbt seed --select raw_clients
dbt snapshot --select scd_client --select scd_client_new
```

Il n'est pas nécessaire d'exécuter `dbt run` pour le modèle de staging car c'est une vue. Il voit donc toujours l'état courant de la table à laquelle il dépend. Une alternative recommandé est de laisser **DBT** gérer l'ordre de reconstruction, tout en lançant les tests :

```
dbt build --select +scd_client --select +scd_client_new
```

La figure 8 montre le SCD type 2 en action.

En faisant une jointure entre `fact_commandes` et `scd_client_new`, il est possible de récupérer la ville du client au moment où il a passé une commande. La requête **SQL** est placé dans le dossier `analyses` :

```sql
select
    c.commande_id,
    c.client_id,
    c.date_commande,
    c.montant_total,
    v.ville        as ville_a_la_commande,
    v.code_postal  as code_postal_a_la_commande,
    v.segment      as segment_a_la_commande,
    d.ville        as ville_actuelle,
    d.segment      as segment_actuel
from commandes c
left join versions v
       on  c.client_id = v.client_id
       and c.date_commande >= v.valid_from
       and c.date_commande <  v.valid_to
left join {{ ref('dim_client') }} d
       on c.client_id = d.client_id
where v.client_id = 2
```

La figure 9 montre le résultat.

Et le SCD de type 1 en DBT ?

Selon les mécanismes **DBT**, une table est une SCD de type 1. Si un enregistrement est modifié dans `seed`, alors le changement sera écrasé partout à travers le DAG (donc dans `stg_produits` et `dim_produits`). Comme les prix unitaires des produits sont présents dans les données brutes des lignes de commandes, le SCD de type 1 se justifie sur le catalogue de produits (pas besoin de conserver l'historique).

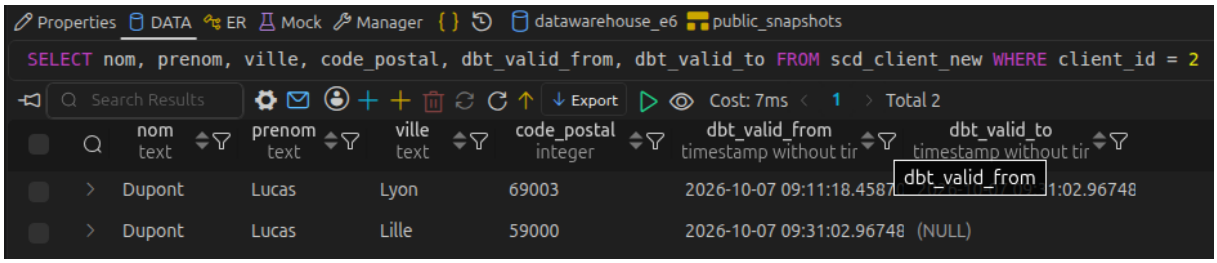


FIGURE 8 – Changement dans le SCD client

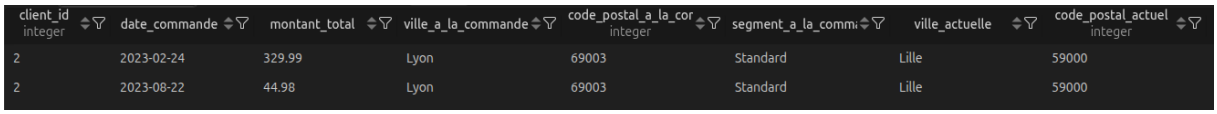


FIGURE 9 – Affichage de l'adresse au moment de la commande.

difficultés rencontrées / liens utiles / pour aller plus loin

6 Scalabilité

6.1 Procédure d'ajout d'un datamart

Un DM (*datamart*) est un "morceau" de DW (*datawarehouse*) regroupant les tables suivant une logique métier. Par exemple un DW peut regroupe un DM Finance, un DM RH, un DM Produits, un DM Ventes, ... Les datamarts étant destinés aux analystes et aux métiers, ils sont plus souvent vues comme le regroupement de tables de faits et de tables de dimensions (et éventuellement de vues ou tables agrégées

pour des usages fréquents). Lorsqu'une table de dimensions fait partie de plusieurs datamarts, elles sont considérées comme conformes.

Le projet **DBT** parcouru dans ce rapport suit l'approche *bottom-up* de Kimball. Des datamarts dimensionnels sont construits suivant un processus métier. On a par exemple le domaine `ventes` dont le DM s'articulerait autour de la table de faits `fact_commandes`, et le domaine `marketing` dont le DM s'articulerait autour de la table de faits `fact_visites`. Ces datamarts sont reliés entre eux formant un entrepôt logique.

Le travail détaillé précédemment autour de l'ajout dans `seed` de **`raw_retours.csv`** n'est autre que l'intégration d'un nouveau DM concernant le domaine `Retour`. En résumé, la procédure pour l'ajouter a été la suivante :

1. Analyse du domaine métier et granularité de la source de données
2. Intégration de la source (`seed`)
3. Normalisation des données (`staging`)
4. Identification des dimensions conformes
5. Création de nouvelles dimensions (si besoin, `dimensions`)
6. Construction de la table de faits en référençant les dimensions (`facts`)
7. Ajout de tests (**`schema.yml`**)
8. Gestion des droits d'accès (**`dbt_project.yml`** et **`schema.yml`**)
9. Mise à jour de la documentation (**`schema.yml`** et **`models.md`**)

Jusqu'à présent, tout a était inséré dans un schéma publique. Une meilleure organisation serait de mettre en place un schéma par couche puis par datamart.

Il existe également une autre approche (*top-down*) de Bob Inmon qui renverse le processus. Selon cette approche, le DW se construit d'abord sur une centralisation des données comme source unique de vérité, puis la création des datamarts de façon périphérique.

liens utiles

6.2 Augmentation de l'espace de stockage

Au rythme où les données affluent dans la base **PostgreSQL**, celle-ci prend du volume. Il arrive un moment où ce volume de la base de données peut devenir critique par rapport à la taille du support de stockage (serveur, VM avec espace alloué, ...). L'idée d'une procédure d'augmentation de l'espace de stockage couvrirait :

1. le suivi régulier silencieux de la taille de la base relativement à l'espace disponible,
2. le système d'alertes lorsque le ratio atteint $X\%$ pour anticiper un besoin de migration ou d'allocation supplémentaire de mémoire, et
3. le lancement d'une procédure de migration lorsque besoin.

Suivi régulier de l'espace de stockage

Dans le conteneur **PostgreSQL**, nous avons déja mis en place une base de données `monitoring` pour le suivi du service. Dans cette base de données, on ajouterait une nouvelle table pour stocker des logs d'un suivi régulier de l'espace de stockage.

En python `shutil.disk_usage(PGDATA)` permettrait d'obtenir les espaces disques du système **PostgreSQL** total, utilisé et libre sur la base desquels on peut établir un ratio. Exécuter toutes les heures, un script incorporant cette commande pourrait enregistrer des mesures de l'espace de stockage.

À noter également que les mesures pourraient intégrer l'usage de `pg_database_size` en **psql**, qui permet de mesurer l'espace disque utilisé par une base de données donnée.

Si le ratio établi est inférieur à, e.g., 70%, rien n'est déclenché. Dans l'autre cas, un avertissement est émis pour permettre de déclencher des opérations : 1) nettoyage de logs, purge RGPD de données, . . . ; 2) préparation d'une migration, planification de l'extension pour se laisser du temps. Si le ratio dépasse 90%, on pourrait passer en mode critique, nécessitant d'opérer une perte de donnée si on veut privilégier la continuité du service, par exemple.

Migration de la base vers un espace plus grand

À ce niveau, la procédure de back-up est réutilisée. Le principe du back-up est précisément de rétablir la base de données. C'est évidemment une opération souhaité lors d'une migration.

Avant toute choise, il serait préférable d'interrompre le service durant l'opération. Cela nécessite de mettre en pause l'afflux des données entrantes. Néanmoins, si le service doit absolument être maintenu (pour respecter le SLA), alors il sera nécessaire de marquer les données arrivée après la dernière opération de backup pour les reproduire après migration.

L'environnement **PostgreSQL** est reproduit sur le nouvel espace plus grand (e.g. dans notre cas pratique, une copie modifiée du docker compose). Les données déchargées durant le back-up sont ensuite chargées dans la nouvelle base via la phase de restauration de la procédure du back-up, en adaptant les hôtes et les ports au besoin.

Puis, si le service a été interrompu, les flux de données entrantes reprennent en étant dupliqués pour pointer vers les deux bases de données alors co-existantes, l'ancienne et la nouvelle. Si le service n'a pas été interrompu, on récupère les données arrivées tardivement pour les répercuter aussi sur le nouvelle base afin de ne pas perdre de données.

Enfin, un protocole de test pour s'assurer de la récupération des données serait ensuite mis en place pour détecter si perte de données il y a eu (e.g. en comparant le nombre de lignes par tables, la présence des données à l'identique après le timestamp du backup, . . .). Après quelques jours de coexistence, si la nouvelle base est validée, l'ancienne base de données peut alors être supprimée.

liens utiles

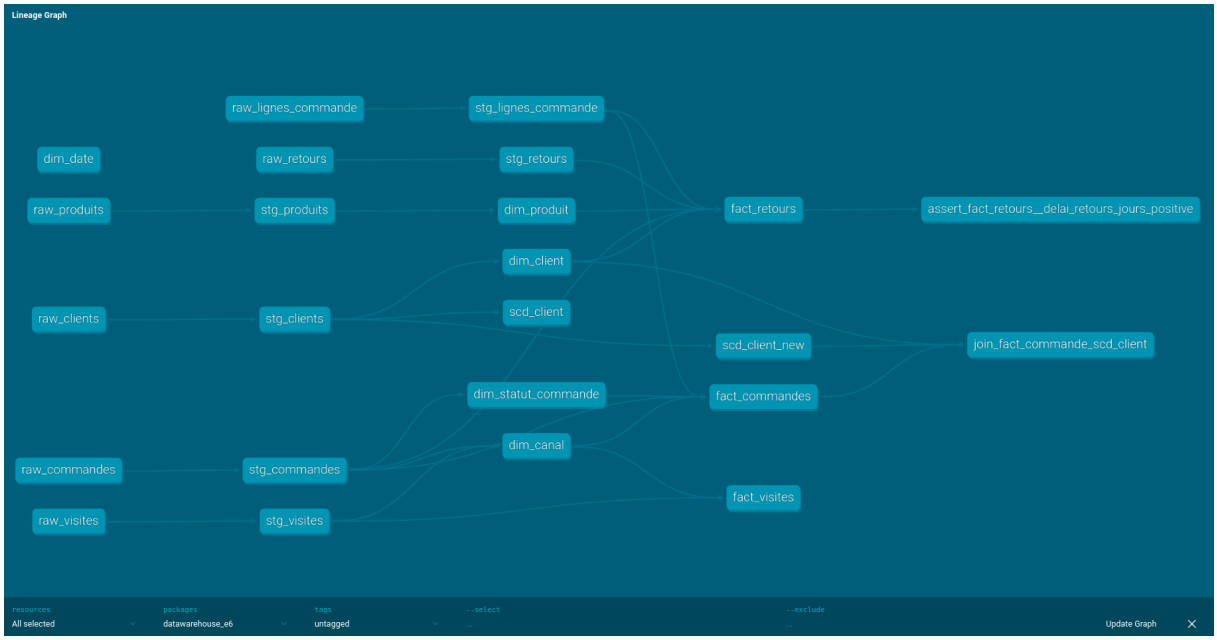


FIGURE 10 – Lignage des données dans l'entrepôt final

7 Conclusion

Ce rapport a couvert l'ensemble des points à traiter lors d'une opération de maintenance portant sur un datawarehouse. Certaines tâches ont été exécutées et le dépôt **github** https://github.com/cesarGattano/Takeoff-DBT permet d'en explorer les résultats. Le lignage final de la base de données est présenté en figure 10 et montre comment les différents ajouts s'insèrent en comparaison avec l'état initial présenté en figure 1.

A Exemple d'une fiche de traitament (RGPD)

Voici un exemple d'une fiche de traitament du registre des traitements qui doit être tenu dans le cadre de la RGPD.

Description du traitement

- **Nom du traitement :** Suivi des retours
- **No/REF :** XXXXX
- **Date de création du traitement :** 5 Octobre 2026
- **Date de mise à jour du traitement :** 6 Octobre 2026

Acteurs

- **Responsable du traitement :** Jean Durant - 1 grande avenue 12345 Patelin France 01.02.03.04.05 jean.durant@unmail.com
- **DPO :** Bob Francis - 1 petite impasse 12345 Patelin France 01.02.03.04.05 bob.francis@unmail.com

Finalités

- **Finalité principale :** Suivi des retours de produits
- **Sous-finalité 1 :** Suivi des remboursements après retour
- **Sous-finalité 2 :** Contrôle qualité des produits post-vente
- **Sous-finalité 3 :** Suivi des délais de retours
- **Sous-finalité 3 :** Communication avec le client

Catégories des données personnelles concernées

- **Données d'identification :** [`dim_client`] nom, prenom, nom_complet, email, ville, code postal, segment
  Durée de conservation : 1 an d'inactivité sur le compte

Catégories des personnes concernées

- **Catégorie de personnes 1 :** client : identification du client ayant passé commande

Destinataires des personnes concernées

- **Destinataire :** service interne qui traite les données

Mesures de sécurité

- **Mesure de sécurité 1 :** Sauvegarde des données - existence de backup contenant les données personnelles
- **Mesure de sécurité 2 :** Contrôle des accès - Limitation d'accès à ces données

B Difficultés rencontrées

B.1 Préparation de la maintenance

B.1.1 À propos du backup régulier

- La base de données **PostgreSQL** tournant dans un conteneur **Docker**, les commandes sont envoyées via un `docker exec`. Certaines options comme `-t` peuvent interférer avec la sortie de

`pg_dump` et corrompre le fichier-dump créé au point que la restauration échoue.

- L'utilisation de `cron` nécessite de s'assurer des droits d'éxecution des différents scripts pour la catégorie `other` ainsi que de l'utilisation de chemins absolus tout au long du script.

B.2 Opération de maintenance

B.2.1 À propos de l'ajout d'un datamart

- Au moment de réaliser ce projet, l'équipe `DBT` est en phase de transition vers `DBT` v2 avec le déploiement de `dbt-fusion` en remplacement de `dbt-core`, mais le connecteur `PostgreSQL` n'est pas encore disponible pour cette version. Bien que la v2 soit encore en pré-release la documentation et l'extension `VSCode` repose sur cette dernière, ce qui a créé une incompatibilité à gérer au niveau de l'installation.

B.2.2 À propos des tests et documentations `DBT`

- Les tests `DBT` ont révélé que certains champs étaient `NULL` . Notamment, les champs date d'expédition et date de livraison, laissant penser qu'il y a peut-être qu'une mode de retrait existe.
- Dans une autre mesure, une donnée avec un `montant_paye` nul semble être de l'ordre de l'erreur.
- Le graphe du lignage de données dans l'UI montre un noeud `dim_date` isolés. Après plusieurs recherches, il s'avère que ce graphe ne se base que sur les balises `ref()` dans les fichiers `SQL` et ne prend pas en compte les relations définies dans les fichiers `schema.yml` . Cet outil montre un DAG (*Direct Acyclic Graph*) incomplet.

B.3 Gestion des accès et procédure d'ajout d'un nouveau rôle

À propos de l'attribution des privilèges

- `dbt build` suppriment les tables pour les récréer ensuite. Tous les privilèges accordés via `psql` sont alors perdus dans le processus et doivent être attribuer à nouveau.

À propos des tests de validation

- Certains tests `Pytest` mis en place ont demandé d'exclure des tables systèmes de `PostgreSQL` pour que les tests passent et ne couvrent que le champ "applicatif" de ce que j'ai mis en place.

B.4 Variation d'une dimension

B.4.1 À propos des SCD dans `DBT`

- Les snapshots fournissent automatiquement les valeurs des champs `dbt_valid_from` , `dbt_valid_to` , `dbt_updated_at` sur l'horodatage actuel. Dans le cadre de l'exercice, ces colonnes sont donc décorellés des données elles-mêmes. La requête `SQL` est de fait plus complexe que ce qui est écrit.

C Liens utiles

C.1 Préparation de la maintenance

C.1.1 À propos du suivi de service

- SLA : https://www.ibm.com/fr-fr/think/topics/service-level-agreement
- Objectifs de service : https://sre.google/sre-book/service-level-objectives/

C.1.2 À propos du backup régulier

- **PostgreSQL** backup : https://www.postgresql.org/docs/current/backup.html
- Data integrité : https://sre.google/sre-book/data-integrity/

C.2 Opération de maintenance

C.2.1 À propos de l'ajout d'un datamart

- **DBT** fundamentals (**VSCode**) : https://learn.getdbt.com/learn/course/dbt-fundamentals-vs analytics-development-lifecycle-30min/dbt-and-the-adlc?page=2

C.2.2 À propos des tests et documentation DBT

- **DBT** fundamentals (building tests) : https://learn.getdbt.com/learn/course/dbt-fundamentals/data-tests-60min/building-tests?page=1
- **DBT** fundamentals (documentation basics) : https://learn.getdbt.com/learn/course/dbt-fundamentals/documentation-40min/documentation-basics?page=1
- repo **DBT** : https://github.com/dbt-labs/dbt
- **DBT**fundamentals (documentation basics) : https://learn.getdbt.com/learn/course/dbt-fundamentals/documentation-40min/documentation-basics?page=1

C.3 Gestion des accès et procédure d'ajout d'un nouveau rôle

C.3.1 À propos de l'attribution des privilèges

- Prise en main de **psql** : https://blog.stephane-robert.info/docs/services/bdd/relationnelles/postgresql/prise-en-main-psql/
- docs **psql**"GRANT" : https://docs.postgresql.fr/11/sql-grant.html

C.3.2 À propos des tests de validation

- **Pytest** : parametrizing tests : https://docs.pytest.org/en/stable/example/parametrize.html
- **PostgreSQL**pg_catalog : https://docs.postgresql.fr/15/catalogs.html
- **PostgreSQL**information_schema : https://www.postgresql.org/docs/current/information-schema.html
- **PostgreSQL**pg_toast : https://docs.postgresql.fr/12/storage-toast.html
- docs **DBT** grants : https://docs.getdbt.com/reference/resource-configs/grants?version=2

C.3.3 À propos du RGPD

- Règlement RGPD : https://www.cnil.fr/fr/reglement-europeen-protection-donnees
- RGPD : registre des activités de traitement : https://www.cnil.fr/fr/definition/registre-des-activites-de-traitement

C.4 Variation d'une dimension

C.5 À propos des SCD dans DBT

- SCD de Ralhp Kimball : https://www.datacamp.com/fr/tutorial/mastering-slowly-changing
- DBT snapshot : https://docs.getdbt.com/docs/build/snapshots?version=2

C.6 Scalabilité

C.6.1 À propos de l'ajout d'une datamart

- Datamarts : https://www.data-bird.co/blog/datamart?utm_source=website&utm_medium=unknown_page&utm_campaign=general&utm_content=cta_formulaire&_utmw=1
- Datamarts : https://liora.io/datamart-tout-savoir
- Approche *bottom-up* : https://liora.io/methode-de-kimball-tout-savoir
- DBT : structure de projet : https://docs.getdbt.com/best-practices/how-we-structure/1-guide-overview?version=2

À propos de l'augmentation de l'espace de stockage

- PostgreSQL : monitoring disk usage : https://www.postgresql.org/docs/8.1/diskusage.html
- PostgreSQL : adminstrator functions : https://www.postgresql.org/docs/8.1/functions-admin.html#FUNCTIONS-ADMIN-DBSIZE

D Pour aller plus loin

D.1 Préparation de la maintenance

D.1.1 À propos du suivi de service

- Mise en place d'une architecture *collecteur* + *visualiseur* en solution industrielle et gratuite grâce à **Prometheus** et **Grafana**.
- Mise en place d'un système d'alertes structuré sur une fenêtre de confirmation (ex : seuil d'une métrique dépassée sur plusieurs mesures consécutives ou p95 supérieur à un seuil sur les 5 dernières minutes).
  Technologies possibles : **SQL** + **smtplib** ou **Prometheus** Alertmanager + **Grafana** Alerting.

D.1.2 À propos du backup régulier

- **pgBackRest** permet la mise en place d'un back up incrémental ou différentiel.
- Pour une stratégie de back-up robuste, la règle d'or est 3-2-1 : 3 copies sur 2 supports différents, 1 support hors site.
- Planification de retention sophistiquée : 14 backups des derniers jours + 1 backup par mois des 6 derniers mois.

D.2 Opération de maintenance

D.2.1 À propos de l'ajout d'un datamart

- Ajout de tests dans le **schema.yml** du dossier `seeds` pour détecter prématurément des fichiers-csv corrompus.
- Explorer **Fivetran!** pour la partie EL si les sources sont externes.
- Recensement des sources dans un fichiers **sources.yaml** si elles sont externes.

D.3 Gestion des accès et procédure d'ajout d'un nouveau rôle

D.3.1 À propos de l'attribution des privilèges

- L'accès granulaire à cause des données personnelles de `dim_client` n'étant pas évident avec une BDD **PostgreSQL**, une alternative plus puissante serait de créer une vue de `dim_client` dans laquelle les données personnelles sont supprimées ou transformées pour respecter le RGPD. Les avantages sont que cette vue intégrerait le lignage et la documentation, l'attribution de privilèges revient alors au niveau de la table entière, ce qui débloque également le `SELECT *` utile pour la découverte agnostique d'un modèle et enfin, il est possible de transformer les données personnelles en données impersonnelles pour que cela soit utilisable sans compromettre le respect RGPD (e.g. âge $\rightarrow$ tranche d'âge, code postal $\rightarrow$ département)

D.3.2 À propos du RGPD

- Principe de minimisation des données (article 5.1.c) : on peut toujours questionner l'utilité de la conservation des données personnelles au regard des traitements établis.
- Dans ce projet, il est également question d'une vue SCD2 `scd_client`. Cette vue répercute également les données personnelles précédemment listées, et doit être intégrés à une procédure de purge.

D.4 Variation d'une dimension

À propos des SCD dans DBT

- la documentation **DBT**recommande également d'avoir un schema dédié pour les snapshots plutôt que de les mélanger aux modèles. L'argument pour cela est de pouvoir conserver l'historique même en cas de drop ou de refresh des modèles (schéma public).
- Les snapshots font partie intégrante d'un projet **DBT**. Comme ils contiennent l'historique, il est impératif de les inclure dans la procédure de backup. Le `scd_client` particulièrement traite de données personnelles. Il doit donc être considéré dans la gestion des accès, ainsi que la procédure de purge mise en place en application de la RGPD.
- Il est reconnu plus efficace d'ajouter une surrogate key à une table de dimension SCD2 et d'inclure la surrogate key dans la table de faits au moment du chargement d'un fait. La jointure est ainsi direct et ne nécessite pas de jointure temporelle, plus coûteuse.